

Ball Machine

Madina ha inventado una máquina lanzapelotas para entretener a los participantes de la IOI. La estructura interna de la máquina es la siguiente:

- La máquina consta de **nodos** N , indexados desde 0 hasta $N - 1$. El nodo $N - 1$ se denomina **raíz**.
- Para cada nodo u con $0 \leq u < N - 1$, el **padre** del nodo u es un nodo $P[u]$ con $P[u] > u$, y el nodo u es un **hijo** de $P[u]$. La raíz no tiene padre. Un nodo sin hijos se denomina **hoja**.

No conoces N ni el padre $P[u]$ de ningún nodo u . En cambio, Madina te dice M , el número de hojas, y que los índices de las hojas son $0, 1, \dots, M - 1$. Tu tarea consiste en determinar la estructura interna de la máquina mediante su funcionamiento.

Cada nodo de la máquina puede contener como máximo una **bola**, y cada bola tiene un **valor** que es un entero no negativo. Decimos que un nodo tiene valor x si contiene una bola con valor x . Un nodo está **ocupado** si contiene una bola; en caso contrario, está **libre**. Inicialmente, todos los nodos están libres.

Puedes realizar dos tipos de operaciones:

- **insert**(U, X): intento de insertar una nueva bola con el valor X en la hoja U .
 - Si el nodo hoja U está ocupado, la operación devuelve `false` sin insertar la bola.
 - Si el nodo hoja U está libre, la bola se coloca en el nodo U . Luego, la bola se desplaza repetidamente hacia el padre de su nodo actual, siempre que dicho nodo padre esté libre. El desplazamiento se detiene cuando la bola llega a la raíz o a un nodo cuyo padre esté ocupado. La operación devuelve `true`.
- **collect**(): si la raíz está libre (lo que significa que la máquina está vacía), la función «collect» devuelve un array vacío. En caso contrario, la función recursiva «traverse», que se describe a continuación, recopila los valores de todas las bolas que se encuentran actualmente en la máquina en un array « S ». Inicialmente, el array « S » está vacío y se llama a la función «traverse($N - 1$)».

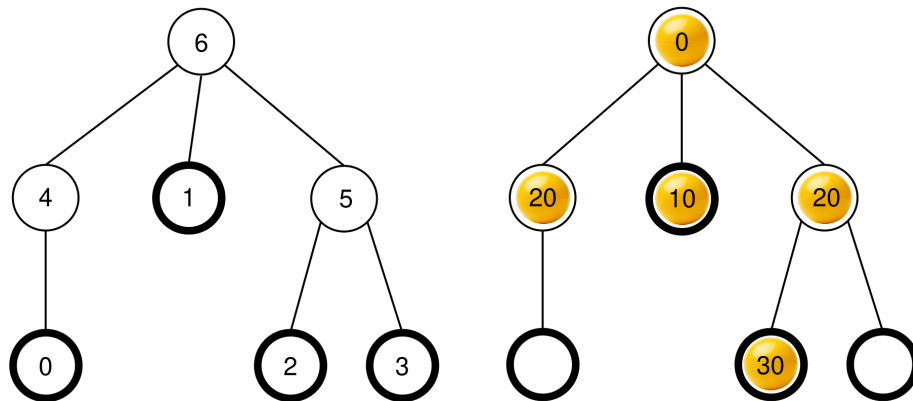
```

traverse(u):
    añadir el valor de la bola en el nodo u al final de S
    sea c[u] la lista de los hijos ocupados de u
    ordena c[u] en orden no decreciente según los valores
    de las bolas en esos nodos (si varios nodos tienen el
    mismo valor, pueden aparecer en cualquier orden)
    para cada v en c[u]:
        traverse(v)

```

Una vez finalizado este procedimiento, **se retiran todas las bolas** de la máquina, y `collect` devuelve S .

Por ejemplo, consideremos una máquina con $N = 7$ nodos y un arreglo padre $P = [4, 6, 5, 5, 6, 6]$. La imagen de la izquierda muestra la máquina vacía con los índices de los nodos, mientras que la de la derecha muestra una posible configuración de las bolas tras una secuencia de operaciones `insert` (esta secuencia se explica en la sección «Example»).



Cuando se llama a `collect()`, la raíz (nodo 6) está ocupada, por lo que S se inicializa como $S = []$ y `traverse(6)` es llamada.

traverse(6): el nodo 6 tiene el valor 0; al agregarlo a S se obtiene $S = [0]$. Los hijos del nodo 6 son 4, 1 y 5, todos los cuales están ocupados. Sus valores son 20, 10 y 20, respectivamente. Ordenando en orden no decreciente se obtiene $c[6] = [1, 5, 4]$ (nótese que $c[6] = [1, 4, 5]$ también sería válido, ya que los nodos 4 y 5 tienen el mismo valor). El procedimiento luego llama a `traverse` en cada nodo en $c[6]$ en ese orden.

- **traverse(1)**: el nodo 1 tiene el valor 10; al añadirlo a S se obtiene $S = [0, 10]$. El nodo 1 no tiene hijos ocupados, por lo que esta llamada finaliza.
- **traverse(5)**: el nodo 5 tiene el valor 20; al añadirlo a S se obtiene $S = [0, 10, 20]$. El nodo 5 tiene un hijo ocupado, el nodo 2, por lo que $c[5] = [2]$ y el procedimiento llama a `traverse(2)`.

- **traverse(2)**: El nodo 2 tiene el valor 30; al añadirlo a S se obtiene $S = [0, 10, 20, 30]$. El nodo 2 no tiene hijos ocupados, por lo que esta llamada finaliza.
- La ejecución vuelve a **traverse(5)**, que ya ha procesado todos los hijos de $c[5]$, por lo que su llamada finaliza.
- **traverse(4)**: El nodo 4 tiene el valor 20; al añadirlo a S se obtiene $S = [0, 10, 20, 30, 20]$. El nodo 4 no tiene hijos ocupados, por lo que esta llamada finaliza.

Todas las llamadas han finalizado y no quedan más nodos por procesar. Por último, se retiran todas las bolas de la máquina y **collect** devuelve el array $S = [0, 10, 20, 30, 20]$.

Tu tarea consiste en determinar el número de nodos N y proporcionar un arreglo de los padres $R = [R[0], R[1], \dots, R[N-2]]$ que describa la estructura de la máquina, pero con una etiquetación potencialmente diferente de los nodos que no sean hojas ni la raíz. Formalmente, un arreglo R proporcionado se considera correcta si es posible asignar una etiqueta distinta $L[u]$ con $0 \leq L[u] < N$ a cada nodo u de la máquina, de tal manera que:

- $L[u] = u$ para todos $0 \leq u < M$ y $u = N-1$, y
- $L[P[u]] = R[L[u]]$ para todos $0 \leq u < N-1$.

No es necesario que $R[u] > u$. La máquina **debe estar vacía** cuando envíes tu solución. Sea K el número de operaciones **collect** realizadas, y B el valor máximo de cualquier bola en todas las llamadas **insert**. Entonces $C = K + B \cdot C$ **no debe superar** 1000. Tu puntuación en algunas subtarefas depende del valor de C .

Implementation Details

Debes implementar el siguiente procedimiento.

```
std::vector<int> find_structure(int M)
```

- M : el número de hojas en la máquina.
- El procedimiento se llama exactamente una vez para cada caso de prueba.
- El procedimiento debe devolver una matriz $R = [R[0], R[1], \dots, R[N-2]]$ de longitud $N-1$, un arreglo correcto de padres.

Para interactuar con la máquina, su procedimiento puede llamar a los dos procedimientos siguientes.

El primer procedimiento es:

```
bool insert(int U, int X)
```

- U : el índice de la hoja donde se debe insertar la bola. Se debe cumplir la condición $0 \leq U < M$.
- X : el valor entero de la bola. Debe cumplirse la condición $0 \leq X \leq 1000$.
- Este procedimiento devuelve `true` si la bola se insertó correctamente, o `false` si la hoja U ya estaba ocupada.
- Este procedimiento puede ser llamado como máximo 500 000 veces.

El segundo procedimiento es:

```
std::vector<int> collect()
```

- Recopila los valores de todas las bolas insertadas, vacía la máquina y devuelve el array recopilado S .

Si en algún momento durante la ejecución de su programa el valor de C supera 1000 , su solución recibirá el veredicto `Output isn't correct: Too many resources used.`

La estructura de la máquina se fija antes de que se llame a `find_structure` . El grader es **deterministic** en el sentido de que si se ejecuta dos veces y ambas ejecuciones realizan la misma secuencia de operaciones, las llamadas a `collect` devolverán los mismos arreglos.

Constraints

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

Subtasks

Subtask	Score	Restricciones adicionales
1	5	La raíz tiene exáctamente M hijos.
2	10	$M \leq 3$
3	25	$N \leq 200, M \leq 45$
4	60	No hay restricciones adicionales.

En las subtareas 3 y 4, su puntuación depende del valor de C de la siguiente manera.

Subtask 3 (25 points)

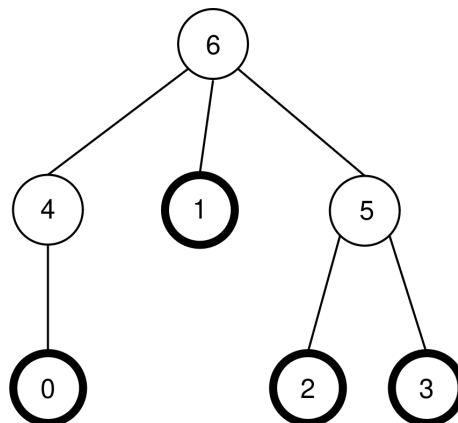
Limits	Score
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

Subtask 4 (60 points)

Limits	Score
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

Ejemplo

Consideremos la misma máquina que en la descripción de la tarea, por lo que $N = 7$, $M = 4$ y $P = [4, 6, 5, 5, 6, 6]$. La máquina se muestra nuevamente en la siguiente figura:



El grader hace la siguiente llamada:

```
find_structure(4)
```

El procedimiento puede realizar la siguiente secuencia de llamadas:

1. **insert(0, 0)** : la hoja 0 está libre, por lo que se coloca una bola con valor 0 en la hoja 0 . El nodo $P[0] = 4$ está libre, por lo que la bola se mueve al nodo 4 . El nodo $P[4] = 6$ está libre,

por lo que la bola se mueve al nodo 6 . Dado que el nodo 6 es la raíz, el movimiento se detiene. La llamada devuelve `true` .

2. `insert(3, 20)` : la hoja 3 está libre, por lo que se coloca una bola con valor 20 en la hoja 3 . El nodo $P[3] = 5$ está libre, por lo que la bola se mueve al nodo 5 . Como $P[5] = 6$ está ocupado, el movimiento se detiene. La llamada devuelve `true` .
3. `insert(1, 10)` : la hoja 1 está libre, por lo que se coloca una bola con valor 10 en la hoja 1 . Como $P[1] = 6$ está ocupado, la bola permanece en el nodo 1 . La llamada devuelve `true` .
4. `insert(2, 30)` : la hoja 2 está libre, por lo que se coloca una bola con valor 30 en la hoja 2 . Como $P[2] = 5$ está ocupado, la bola permanece en el nodo 2 . La llamada devuelve `true` .
5. `insert(1, 25)` : la hoja 1 está ocupada, por lo que la bola no se coloca en la hoja 1 . La llamada devuelve `false` .
6. `insert(0, 20)` : la hoja 0 está libre, por lo que se coloca una bola con valor 20 en la hoja 0 . El nodo $P[0] = 4$ está libre, por lo que la bola se mueve al nodo 4 . Como $P[4] = 6$ está ocupado, el movimiento se detiene. La llamada devuelve `true` .

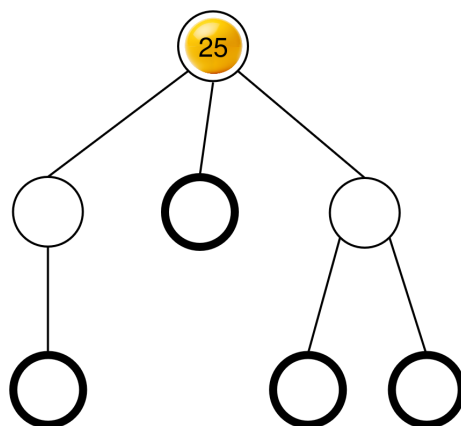
La configuración resultante de las bolas ya se mostró en la descripción de la tarea.

A continuación, el procedimiento puede llamar a:

```
collect()
```

La ejecución de esta llamada ya se explicó en la descripción de la tarea, por lo que esta llamada devuelve $S = [0, 10, 20, 30, 20]$. Después, la máquina vuelve a estar vacía.

A continuación, el procedimiento puede llamar a `insert(2, 25)` . La hoja 2 está libre, por lo que se coloca una bola con valor 25 en la hoja 0 . Esta bola se mueve luego al nodo 5 y luego al nodo 6 , donde se detiene. La llamada devuelve `true` . La configuración resultante de bolas se muestra en la siguiente figura.



Finalmente, el procedimiento puede llamar a `collect()` que devuelve $S = [25]$ y vacía la máquina.

Hay dos arreglos de padres correctos que `find_structure` puede devolver: $R = P = [4, 6, 5, 5, 6, 6]$ (que corresponde al etiquetado $L = [0, 1, 2, 3, 4, 5, 6]$), y $R = [5, 6, 4, 4, 6, 6]$ (que corresponde al etiquetado $L = [0, 1, 2, 3, 5, 4, 6]$). En este ejemplo, $K = 2$ y $B = 30$, por lo que $C = 32$.

Sample Grader

Input format:

```
N M
P[0] P[1] P[2] ... P[N-2]
```

Output format:

```
K B C
Q
R[0] R[1] R[2] ... R[Q-1]
```

Aquí, Q es la longitud del array R devuelto por `find_structure`.